



APM: Agent Package Manager

Dependency management for AI agent configuration

20 Minutes

Technical Audience

[microsoft/apm](#)

Open Source

Application Code ✓

-  `package.json` — declare dependencies
-  `package-lock.json` — pin versions
-  `npm install` — reproducible setup
-  Transitive dependency resolution
-  `npm audit` — security scanning

Solved decades ago. Every language has one.

Agent Configuration ✗

-  Copy instruction files between repos
-  Write prompts from scratch every project
-  Configure MCP servers per-developer
-  Hope everyone's setup matches
-  No audit trail for agent config

5 devs = 5 divergent configs. 50 devs = 50.

Think `package.json` for AI agent configuration. Declare, resolve, lock, distribute.

The Manifest: `apm.yml`

```
name: my-project
version: 1.0.0
dependencies:
  apm:
    - microsoft/apm-sample-package
    - anthropics/skills/frontend-design
    - github/awesome-copilot/agents/api-architect
```

```
$ apm install → Done. All configured.
```

What You Get

 **Transitive dependencies** resolved automatically

 **Lock file** pins every dep to exact commit

 **Any git host** — GitHub, GitLab, Bitbucket, Azure DevOps

 **Security scanning** before deployment

 **Audit trail** — every change tracked, diffable in PRs

APM manages seven types of agent configuration. Each is a first-class citizen in the manifest and dependency tree.



Instructions

Standards & guardrails



Skills

Reusable capabilities



Prompts

Slash commands



Agents

Specialized personas



Hooks

Lifecycle handlers



Plugins

Packaged bundles



MCP Servers

Tool integrations



All in one

`apm.yml`

These map directly to the configuration surfaces of major AI coding tools. APM manages what already exists — it doesn't invent new abstractions.

CONSUME**COMPOSE****LOCK****BUILD****DISTRIBUTE** **Consume**

Install from any git host.
Resolve full dependency tree.

```
apm install org/pkg
```

 Compose

Combine org + team +
project config in one
manifest.

```
apm.yml
```

 Lock

Pin every dep to exact
commit. Deterministic
installs.

```
apm.lock.yaml
```

 Build

Compile to native formats
for each AI tool.

```
apm compile
```

 Distribute

Push to git. No registry
needed. Or `apm pack` for
offline.

```
git push / apm pack
```

Marketplaces handle **discovery & distribution**. APM adds **governance, composition & reproducibility**. They complement each other.

Capability	Marketplaces	APM	Together
Browse & discover plugins	✓	—	✓
One-command install	✓	✓	✓
Cross-tool plugin format	✓	✓	✓
Version locking (exact commit)	✗	✓	✓
Transitive dependency resolution	✗	✓	✓
Multi-source composition	✗	✓	✓
Security scanning (Unicode, bidi)	✗	✓	✓
CI/CD reproducibility	✗	✓	✓
Audit trail (lock file in PRs)	✗	✓	✓

Marketplace → APM Workflow

```
# Register a marketplace
$ apm marketplace add github/awesome-copilot

# Browse available plugins
$ apm marketplace browse awesome-copilot

# Install with full APM treatment
$ apm install code-review@awesome-copilot

# Result: locked, scanned, tracked
```

Provenance tracked in lock file:

```
discovered_via: awesome-copilot
resolved_commit: abc123def...
```

The Architecture Stack

AI Coding Tools

Copilot · Claude · Cursor · OpenCode · Codex · Gemini



Plugin / Extension Systems

Marketplaces — discovery & distribution



APM

Dependency mgmt · composition · lock files · audit



Git

Source of truth & distribution

`apm install` deploys native formats. `apm compile` bridges the rest. **Zero lock-in** — stop using APM and your files still work.

AI Tool	<code>apm install</code> deploys	<code>apm compile</code> adds	Support
GitHub Copilot	<code>.github/</code> instructions, prompts, agents, hooks, plugins, MCP	<code>AGENTS.md</code>	Full
Claude	<code>.claude/</code> commands, skills, MCP	<code>CLAUDE.md</code>	Full
Cursor	<code>.cursor/rules/</code> , <code>.cursor/agents/</code> , skills, hooks, MCP	<code>.cursor/rules/</code>	Full
OpenCode	<code>.opencode/</code> agents, commands, skills, MCP	Via <code>AGENTS.md</code>	Full
Codex CLI	—	<code>AGENTS.md</code>	Compile
Gemini	—	<code>GEMINI.md</code>	Compile

Why Agent Config ≠ App Config

Traditional package managers have a **gap** between install and execute.

```
npm install → review time → npm start
```

Agent configuration has **no such gap**.

```
file lands in .github/prompts/ → instantly  
ingested
```

File presence IS execution. No review window.

APM's Pre-Deployment Gate

- 1 **Download** — package cached in `apm_modules/`
- 2 **Scan** — hidden Unicode, bidi overrides, tag chars
- 3 **Deploy** — only if clean → agent-readable dirs
- 4 **Hash** — SHA-256 stored in `apm.lock.yaml`

Critical findings block deployment. Package available for inspection but never reaches agent dirs.

CI-ready: `apm audit -f sarif -o audit.sarif` for GitHub Code Scanning

Three Modes

Mode	Manifests	Use When
APM-only	<code>apm.yml</code>	Full APM workflow
Plugin-only	<code>plugin.json</code>	Standard plugin, no APM needed
Hybrid	Both	Author with APM, export as plugin

Hybrid Workflow

```
# Initialize with plugin support
$ apm init my-plugin --plugin

# Add dev-only deps (excluded from export)
$ apm install --dev owner/helpers

# Export as standard plugin
$ apm pack --format plugin
```

Plugin Detection (Priority Order)

1st `plugin.json` (root)

2nd `.github/plugin/plugin.json`

3rd `.claude-plugin/plugin.json`

4th `.cursor-plugin/plugin.json`

Cross-compatible: A single plugin works in Copilot CLI, Claude Code, and Cursor — APM normalizes the format.



Solo / Small Team

"I use Copilot AND Claude. The project needs 5 plugins."

Without APM: 5 install commands, hope versions match

```
apm install → done
```



Mid-size Team

"We have org-wide security standards, team plugins, and project config."

`apm.yml` composes all three layers via dependency resolution

```
apm.lock.yaml → same setup everywhere
```



Enterprise

"When security asks 'what agent config was active when release 4.2.1 shipped?'"

```
git log apm.lock.yaml
```

 answers that

Versioned · Auditable ·
Reproducible

15 Minutes — Try It

```
curl -sSL https://aka.ms/apm-  
unix | sh  
apm init my-project  
apm install microsoft/apm-  
sample-package
```

-  See files deploy to `.github/`
-  Check the generated `apm.lock.yaml`
-  Run `apm audit`

1 Hour — Team Setup

-  Create `apm.yml` with your org standards + team config
-  Add a marketplace: `apm marketplace add github/awesome-copilot`
-  Install from marketplace: `apm install plugin@marketplace`
-  Commit `apm.lock.yaml` to your repo
-  Run `apm compile` for multi-tool output

2-4 Hours — Enterprise

-  Set up `apm audit` in CI pipeline (SARIF output)
-  Create org-wide packages for shared standards
-  Build a private marketplace for your org
-  Use `agentrc` to auto-generate instructions
-  Review `apm.lock.yaml` diffs in PRs

APM Documentation

 [APM Documentation — Complete Reference](#)

 [microsoft/apm — GitHub Repository](#)

 [Quick Start — Setup in Under a Minute](#)

 [Enterprise Security Model](#)

Related Resources

 [agentrc — Auto-generate Instructions from Code](#)

 [AGENTS.md — Open Standard](#)

 [Agent Skills — Open Standard](#)

 [Companion Talk: Copilot Marketplaces — Discovery & Distribution](#)



Agent Config Is Infrastructure

Declare it. Lock it. Audit it. Ship it.

[microsoft/apm](#)

[Open Source](#)

```
curl -sSL https://aka.ms/apm-unix | sh && apm install
```
